

ARCHITECTURE DECISION LOG — HACKMYSURU 1.0

Project: Civic Follow-through — Clean Mysuru

Team: Adinath, Jai Ganesh, Aakash, Vedanth

Date: 20 September 2026

1. Backend & Database

Context: Complaints require structured storage, history and risk data.

Alternatives: Direct database access, NoSQL, relational API architecture.

Decision: We chose **React/Vite + Express + Supabase PostgreSQL**, with the backend owning validation, persistence and database access.

Trade-off: More API complexity, but better consistency and separation of responsibilities.

2. Risk Engine

Context: Complaints need early risk detection with understandable reasons.

Alternatives: ML, LLM prediction, rules-based engine.

Decision: Vedanth implemented a **Python rules-based risk engine** producing a 0–100 score, risk level and explanations from complaint age, inactivity, priority, workload and history.

Trade-off: Less adaptive than ML, but deterministic, testable and explainable with limited data.

3. Complaint Lifecycle

Context: Current status alone cannot prove follow-through.

Alternatives: Status only, single update field, status history.

Decision: We implemented **SUBMITTED → ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → RESOLVED/REJECTED**, recording transitions with timestamps and remarks.

Trade-off: More database structure, but provides an auditable timeline and data for risk analysis.

Team responsibilities: Jai Ganesh—frontend; Aakash—backend/database; Vedanth—risk engine/testing; Adinath—architecture/integration/QA.